

# Speech and language technology (DA 3452)

...

Lecture 4 – Aug 11  
Concluding BPE, Bag-Of-Words, TF-IDF

# Module 1

## Representing Language Computationally

Tokenization

# Module 1: Where we are

1 The Text Representation Problem

2 Characters and Unicode

3 **Normalized Text → Subword Tokens**

4 Tokens → Document Vectors

# Recap : Visible text and Code point sequence are different

STRING A

c a f é

U+0063 U+0061 U+0066 U+00E9

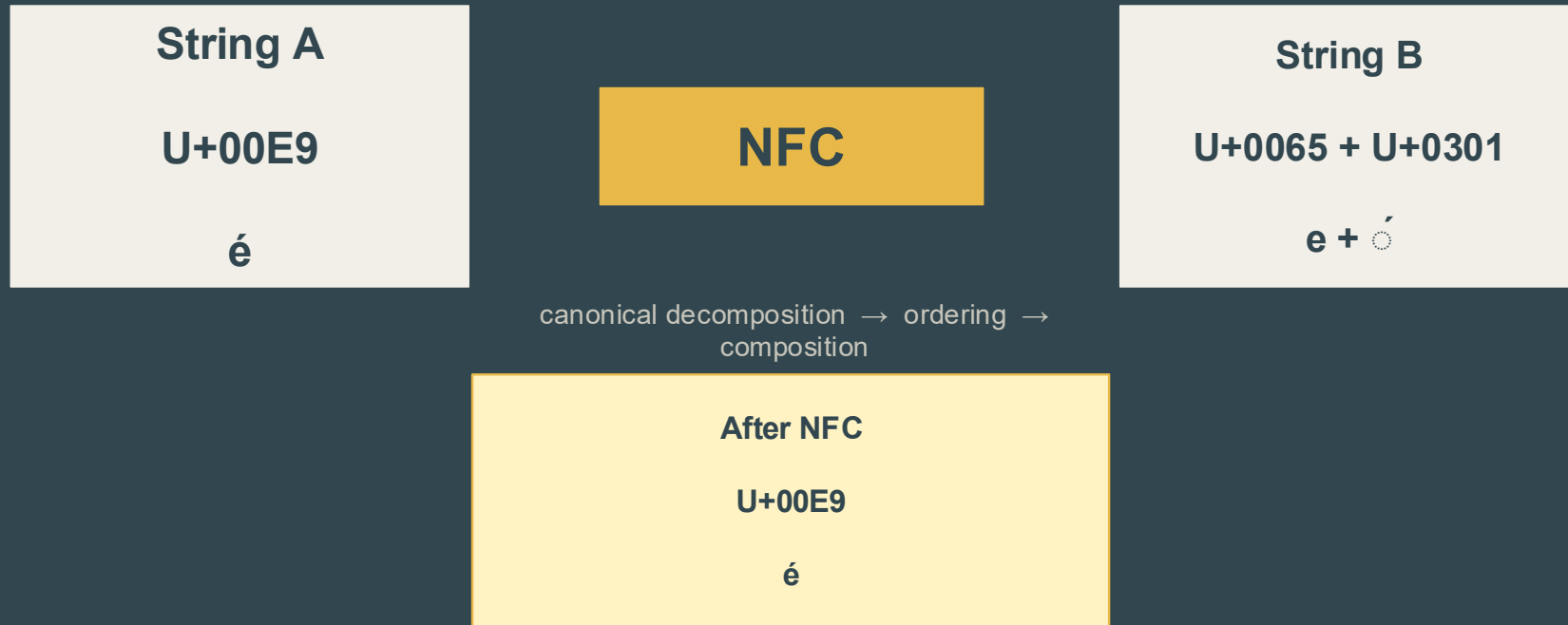
STRING B

c a f e ó

U+0063 U+0061 U+0066 U+0065 U+0301

Visible text  $\neq$  code-point sequence  $\neq$  computational identity

# Recap : NFC Normalization



NFC makes canonically equivalent strings compare the same, count the same, and tokenize the same.

# Recap : NFC normalization in Python

## Code

```
import unicodedata

a = "caf\u00E9"      # precomposed
b = "cafe\u0301"     # decomposed

print(a == b)
print(len(a), len(b))

a = unicodedata.normalize("NFC", a)
b = unicodedata.normalize("NFC", b)

print(a == b)
print([f"U+{ord(c):04X}" for c in b])
```

## Output

```
False
4 5
True
['U+0063', 'U+0061',
 'U+0066', 'U+00E9']
```

## What changed?

- The displayed string did not change.
- Equality changed: before NFC → False; after NFC → True.
- The representation is now stable for counting and tokenization.

# Recap : Two meanings of normalization

## UNICODE NORMALIZATION

**Standardize equivalent representations**

é ↔ e + é

Preserves the intended abstract text

## TASK-SPECIFIC NLP NORMALIZATION

**Choose which distinctions to remove**

Apple → apple

I'm → I am

₹1,000 → 1000

um → [removed]

**Task normalization is a modelling decision: what information are we willing to discard?**

# Recap : Normalized text is still not **tokens**

Even after normalization, what should the model units be?

Words

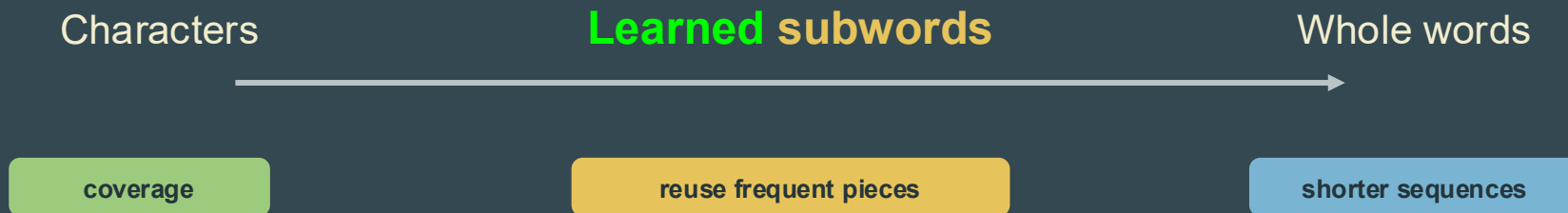
Characters

Morphemes

Learned subwords



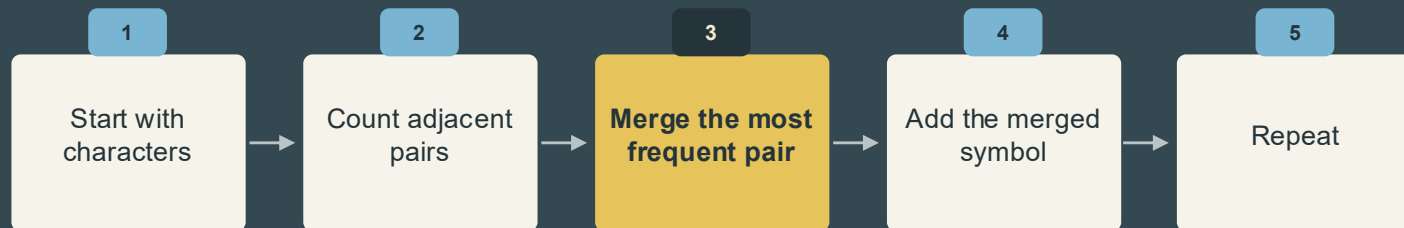
# Recap : Learned subwords



low lower lowest → low + er | low + est

Tokenization keeps frequent strings intact; breaks rare strings into reusable pieces.

# Recap : The BPE Algorithm



**function** BYTE-PAIR ENCODING(strings  $C$ , number of merges  $k$ ) **returns** vocab  $V$

```
 $V \leftarrow$  all unique characters in  $C$            # initial set of tokens is characters
for  $i = 1$  to  $k$  do                           # merge tokens  $k$  times
     $t_L, t_R \leftarrow$  Most frequent pair of adjacent tokens in  $C$ 
     $t_{NEW} \leftarrow t_L + t_R$                  # make new token by concatenating
     $V \leftarrow V + t_{NEW}$                        # update the vocabulary
    Replace each occurrence of  $t_L, t_R$  in  $C$  with  $t_{NEW}$  # and update the corpus
return  $V$ 
```

# Recap : Training a tokenizer versus using it

## TRAINER (done once)

Training corpus + initial symbols +  
vocabulary budget



Tokenizer trainer



**Learned tokenizer**  
vocabulary + ordered merge rules

Example learned from corpus

Vocabulary additions: `ne`, `new`, `re`, ...

Merge order: 1. `n+e→ne` 2. `ne+w→new` 3. `r+e→re`

The vocabulary and rules are learned here.

vs

## ENCODE (each input)

New normalized text + fixed learned  
tokenizer



Token strings



Token IDs

Example encoding

`newest` → `[new]` `[e]` `[s]` `[t]`

`[417]` `[68]` `[84]` `[85]` illustrative IDs

No new tokens or rules are learned here.

**Training learns the tokenizer. Encoding applies the fixed tokenizer to new text.**

An unseen word is represented using existing tokens; it is not automatically added to the vocabulary.

# Recap : BPE training example: initial corpus

**Corpus: set new new renew reset renew**

|       |    |           |
|-------|----|-----------|
| new   | x2 | n e w     |
| renew | x2 | r e n e w |
| set   | x1 | s e t     |
| reset | x1 | r e s e t |

| corpus           | vocabulary                |
|------------------|---------------------------|
| 2    _ n e w     | _ , e , n , r , s , t , w |
| 2    _ r e n e w |                           |
| 1    s e t       |                           |
| 1    _ r e s e t |                           |

**Question: which adjacent pair is most frequent? Notice that there is a tie (which we break with some preset rules)**

# Recap: The BPE training example

| Step | Selected pair | New token | Added to V |
|------|---------------|-----------|------------|
| 1    | n + e         | ne        | {ne}       |
| 2    | ne + w        | new       | {new}      |
| 3    | r + e         | re        | {re}       |
| 4    | s + e         | se        | {se}       |
| 5    | se + t        | set       | {set}      |

## Ordered merge rules

1. **n + e** → **ne**
2. **ne + w** → **new**
3. **r + e** → **re**
4. **s + e** → **se**
5. **se + t** → **set**

## Tokenizer state after 5 merges

Initial symbols: {e, n, r, s, t, w}

Added tokens: {ne, new, re, se, set}

**Vocabulary = initial symbols + added tokens**

Each iteration updates the corpus, adds one vocabulary item, and records one ordered merge rule.

# Recap : Encoding an unseen word

**newest**

Start

n e w e s t

Apply  $n+e \rightarrow ne$

ne w e s t

Apply  $ne+w \rightarrow new$

new e s t

Final tokens

[new] [e] [s] [t]

The whole word was **unseen**—but it can still be encoded.

## Module 1

# Representing Language Computationally

Continuing BPE

# Subwords are not necessarily morphemes

newest  $\rightarrow$  [new] [e] [s] [t]

not automatically

[new] [est]

BPE only learns frequent symbol pairs within the corpus. It does not automatically detect linguistic morphology.



# Characters vs Bytes in BPE

## Classroom example: character-level BPE

- Start with characters
- Easy to read and work through by hand
- Character inventory depends on corpus

Our example

**new** → **n** **e** **w** → ...

v  
s

A non-ASCII character here is one initial symbol  
**é** is treated as a character-level  
symbol

## Many practical tokenizers: byte-level BPE

- Start with UTF-8 bytes
- Fixed initial inventory of 256 byte values
- Guarantees coverage of all Unicode text

Unicode text

↓ UTF-8

byte sequence

↓ BPE merges

tokens

**é** → **UTF-8 bytes** **C3 A9**  
so the initial symbols are bytes, not Unicode  
characters

**The BPE algorithm works identically with characters or bytes. Many LLMs use bytes in BPE**

# Choosing k : Vocabulary size versus sequence length

## SMALL VOCABULARY

- ✓ smaller embedding table
- ✓ strong coverage
- ✗ longer sequences
- ✗ less text per context window

## LARGE VOCABULARY

- ✓ shorter sequences
- ✓ frequent strings become units
- ✗ larger embedding/output layers
- ✗ many rare tokens

Tokenizer design decides where the system pays: vocabulary size or sequence length.

# Multilingual tokenization and token fertility

**Token fertility** = number of tokens used for a word or fixed span of text

## LOWER FERTILITY

“language technology”

[language] [technology]

## HIGHER FERTILITY

“மொழி தொழில்நுட்பம்”

[மொ] [ழி] [தொ] [ழி] [ல்] [நு] [ட்] [பம்]

*Illustrative, tokenizer-specific example*

# Why token fertility matters

## More tokens

for the same content

## Less text

fits in a fixed context

## More computation

during training and inference

## Harder long-range modelling

for the same linguistic span

Tokenization choices can have very important downstream effects

# Module 1

## Representing Language Computationally

Document Vectors

# Why a document-level representation?

normalized text

*language models learn from language data ...*

## SEQUENCE VIEW

Preserve the order of tokens

$[t_1] [t_2] [t_3] \dots [t_n]$



$[417] [68] [84] \dots$

Useful for: next-token prediction, sequence modelling, modern language models

## DOCUMENT VIEW

Summarize the whole text

fixed set of features



$\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{|V|}]$

Good for: search, comparison, document classification

A token **sequence** is **ONE representation of text**; But is it the best representation for every task?

# Why not pure token system for whole document tasks ?

A token sequence is usable—but awkward for many whole-document tasks.

## 1. Different lengths

$D_1 \rightarrow [12, 4, 91, 17]$   
 $D_2 \rightarrow [7, 8, 45, 91, 6, 2, \dots]$   
 $D_3 \rightarrow [\dots 150 \text{ tokens } \dots]$

Documents produce sequences of different lengths.

## 2. IDs are labels

417 = "new"  
68 = "e"

417 is not larger, more important, or semantically "farther" than 68.

## 3. Whole-document questions

What is the topic of the document?  
Which document matches this query?  
Which two documents are similar?

These questions treat the whole document as one object.

Useful target form:

$D_1 \rightarrow [x_1, x_2, \dots, x_{|V|}]$     $D_2 \rightarrow [x_1, x_2, \dots, x_{|V|}]$     $D_3 \rightarrow [x_1, x_2, \dots, x_{|V|}]$

same feature space, same coordinate meanings, regardless of document length

A BPE sequence is valid, but document lengths vary and token IDs are categorical labels.  
If we want to compare, score, or classify documents, a fixed feature space is often useful.

# Motivation for document vectors – Document Search System

## DOCUMENT-LEVEL TASK

A search system begins with:

A collection of documents

A user query

A way to compare query and documents

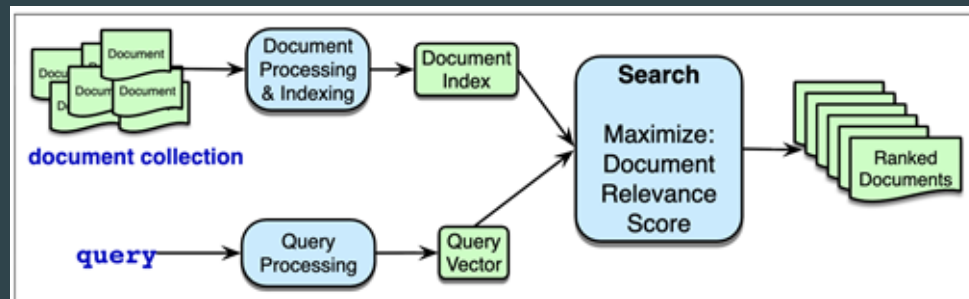
That comparison becomes easier if:

Each document has a common representation

The query has a compatible representation

One classical solution:

Bag-of-words (BoW) + TF-IDF vector



**Figure 11.1** The architecture of an ad hoc IR system. Document ranking is based on computing a score for each candidate document given the query, expressing how **relevant** it is likely to be to meet the users information need. There are two classes of IR systems, based on the two classes of vectors that are used to represent queries and documents: sparse vectors and dense vectors. These two kinds of retrieval differ in the details of the indexing and scoring mechanisms.

We now study one classical way to convert a whole document into a fixed-length vector – BoW + TF-IDF



# Which documents are talking about the same thing?

Intuitively – which texts belong together?

D<sub>1</sub>

the batsman hit the ball  
to the boundary

cricket

D<sub>2</sub>

the bowler bowled the ball  
at the batsman

cricket

D<sub>3</sub>

the vendor sold sweet  
mangoes  
in the market

mango market

D<sub>4</sub>

the price of mangoes rose  
in the market

mango market

What **measurable** properties can we use to recover this property?

# Bag of Words (BoW)

## Bag-of-Words (BoW)

Represent a document by which selected features occur and how often.

“Words” are convenient here; we could also count BPE tokens or characters.

Example Sentence : “the batsman hit the ball to the boundary”

Note that only the **counts matter** but **order is not accounted for**

### 1. Ordered tokens

the batsman hit the ball to the boundary

### 3. Counts

|          |   |
|----------|---|
| the      | 3 |
| batsman  | 1 |
| hit      | 1 |
| ball     | 1 |
| to       | 1 |
| boundary | 1 |

### 4. Fixed feature order → vector

[the, batsman, hit, ball, to, boundary]

$D_1 \rightarrow [3, 1, 1, 1, 1, 1]$

### 2. Unordered Bag

ball the boundary the  
batsman the to hit

# Turning document similarity into a geometric problem

|                                            |         |
|--------------------------------------------|---------|
| DOC 1                                      | CRICKET |
| “the batsman hit the ball to the boundary” |         |

|                                             |         |
|---------------------------------------------|---------|
| DOC 2                                       | CRICKET |
| “the bowler bowled the ball at the batsman” |         |

|                                               |        |
|-----------------------------------------------|--------|
| DOC 3                                         | MARKET |
| “the vendor sold sweet mangoes in the market” |        |

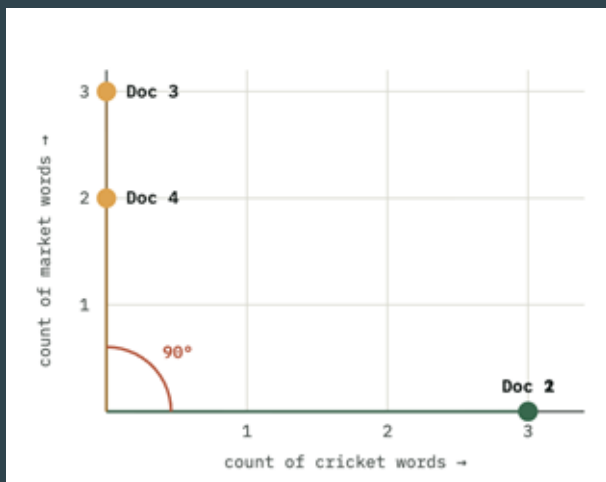
|                                           |        |
|-------------------------------------------|--------|
| DOC 4                                     | MARKET |
| “the price of mangoes rose in the market” |        |

For intuition, compress the real vocabulary-sized space to two grouped axes – **cricket & market**

$D_1$  and  $D_2$  point in similar directions.

$D_3$  and  $D_4$  point in similar directions.

Cricket and market documents point in very different directions.



## Cosine similarity:

“How similar are the directions of two document vectors?”

same direction → cosine  $\approx 1$

right angle → cosine  $\approx 0$

For intuition only — we will return to vector similarity later.

$$\cos(d_1, d_2) = (d_1^T d_2) / (\|d_1\| \|d_2\|)$$

Coordinates here are counts of cricket words (batsman, bowler, ball, boundary) and market words (vendor, mangoes, market) – a 2-D shadow of the real, vocabulary-sized space.